

Kucucikan Laundry POS - System Documentation

Version: 2.8
Last Updated: 2026-05-19
Status: Production Ready
Runtime: Google Apps Script (V8)
Database: Google Sheets (10 sheets)

Table of Contents

- 1. [System Overview](#)
- 2. [Architecture](#)
- 3. [Core Components](#)
- 4. [Data Flow](#)
- 5. [Performance Optimization](#)
- 6. [Security & Reliability](#)
- 7. [Database Schema](#)
- 8. [API Reference](#)
- 9. [Deployment Guide](#)
- 10. [Troubleshooting](#)

System Overview

Purpose

Kucucikan Laundry POS is a cloud-native point-of-sale system designed for small-to-medium laundry businesses (1-3 cashiers). It eliminates infrastructure costs by leveraging Google Workspace ecosystem while maintaining professional-grade features like shift management, audit logging, and real-time analytics.

Key Characteristics

- **Zero Infrastructure Cost:** Runs entirely on Google Workspace (Sheets + Apps Script)
- **Production Ready:** Multi-layer caching, profiling instrumentation, atomic writes
- **Accountability:** Shift management, audit logs, auto-backup to Drive
- **Performance:** Optimized for sub-2s dashboard load, <500ms cold start
- **Accessibility:** WCAG AA compliant, role-based access control

Target Users

- UMKM laundry operators with 1-3 active cashiers
- Business owners requiring real-time monitoring
- Multi-shift operations needing detailed reconciliation

Architecture

Three-Layer Design

CLIENT LAYER (Browser)
- index.html + JavaScript.html + CSS.html + Tailwind.html
- Single-page application with local state management

- 4-layer caching: RAM + sessionStorage + localStorage
- Smart polling with visibility detection

↕ google.script.run

SERVER LAYER (Google Apps Script V8)

- 70+ functions: auth, CRUD, shift mgmt, profiling
- LockService for atomic writes
- CacheService: 4-10 minute TTLs per resource
- Profiling wrapper (_perf) with Stackdriver logging

↕ Sheets API

DATABASE LAYER (Google Sheets)

- 8 active sheets: users, packages, transactions, shifts, customers, kas_awal, pengeluaran, settings
- 2 log sheets: error_logs, perf_logs (auto-rotate)
- Relational schema with 23-column transactions table

Tech Stack

Layer	Technology	Purpose
Backend	Google Apps Script (V8)	Serverless business logic
Database	Google Sheets	Relational data persistence
Frontend	HTML5 + JavaScript ES2020	Single-page application
Styling	Tailwind CSS v4.2.3	Utility-first CSS framework
Charts	ApexCharts (lazy-loaded)	Data visualization
PDF Export	jsPDF + jsPDF-AutoTable	Client-side report generation
Concurrency	LockService	Atomic write operations
Caching	CacheService + sessionStorage + localStorage	Multi-layer caching
Profiling	Custom _perf wrapper	Performance instrumentation
Deployment	clasp CLI	Local ↔ GAS synchronization

Core Components

1. Authentication & Session Management

Token-Based Session System:

- UUID tokens with 8-hour TTL
- Salted password hashing (format: salt:hash)
- Auto-migration from legacy password formats
- Rate limiting: 5 failed attempts → 15-minute logout

- Rolling refresh on each request

Key Functions:

- `login(username, password)` - Verify credentials, issue token
- `validateSession_(token)` - Check token validity, refresh TTL
- `validateAdminSession_(token)` - Guard admin-only endpoints

2. Transaction Management

Multi-Item Support:

- Single transaction can contain multiple items (kiloan + satuan)
- Items stored as JSON array in `items_json` column
- Automatic price lookup via cached price map
- Server-side validation of totals, discounts, change

Payment Methods:

- Tunai (Cash)
- Transfer (Bank transfer)
- QRIS (QR code payment)

Payment Status:

- Lunas (Fully paid)
- Belum Lunas (Partial/pending)
- Down Payment (DP) + Settlement (Pelunasan) tracking

Status Lifecycle:

Proses (Processing) → Selesai (Completed) → Diambil (Picked up)

Key Functions:

- `createTransaction(token, data)` - Create with LockService + batch write
- `updateTransactionStatus(token, id, status)` - Change status
- `lunasDanAmbil(token, id, metode)` - Settlement + shift attribution
- `deleteTransaction(token, id)` - Admin-only deletion

3. Shift Management

Shift Lifecycle:

1. **Open** - Kasir inputs initial cash (`modal_awal`)
2. **Operate** - Transactions auto-tagged with `shift_id`
3. **Close** - Compute summary, persist `breakdown_json`
4. **Force-Close** - Admin override for abandoned shifts
5. **Auto-Close** - Daily 23:50 trigger as fail-safe

Breakdown Persistence:

```
{
  "breakdownMetode": {
    "Tunai": { "dp": 50000, "pelunasan": 30000, "total": 80000, "count": 3 },
    "QRIS": { "dp": 0, "pelunasan": 0, "total": 0, "count": 0 },
    "Transfer": { "dp": 0, "pelunasan": 0, "total": 0, "count": 0 }
```

```

    },
    "breakdownPengeluaran": {
      "Bahan Baku": { "jumlah": 12000, "count": 1 },
      "Operasional": { "jumlah": 0, "count": 0 },
      "Lain-lain": { "jumlah": 0, "count": 0 }
    },
    "totalDp": 50000,
    "totalPelunasan": 30000
  }
}

```

Key Functions:

- `openShift(token, modalAwal)` - Begin shift
- `closeShift(token, shiftId, catatan)` - End shift + persist summary
- `forceCloseShift(token, shiftId, catatan)` - Admin force-close
- `getAllActiveShifts(token)` - Live monitoring (15s cache)
- `getShiftHistory(token, start, end)` - Historical data

4. Cash Management (Manajemen Kas)

Scope Resolution:

- **Kasir + Shift Active** → `shift` scope (window: shift start → now)
- **Kasir + No Shift** → `none` scope (form disabled, warning banner)
- **Kasir + Past Date** → `date` scope (read-only history)
- **Admin** → `date` scope (all shifts, form active)

Total Penerimaan (v2.6+):

- Automatically calculated from DP + Settlement
- Breakdown: Tunai vs Non-Tunai
- Scope-aware: shift window vs calendar date
- Real-time subtitle: "Tunai Rp X • Non-Tunai Rp Y"

Key Functions:

- `getKasHarian(token, tanggalStr)` - Daily cash with scope resolution
- `saveUangAwal(token, nominal, dateStr)` - Record initial cash
- `savePengeluaran(token, keterangan, kategori, jumlah, dateStr)` - Log expense
- `getKasPeriode(token, start, end)` - Multi-day cash recap

5. Reporting & Analytics

Report Data (v2.8+):

- Server-side pre-aggregation: `byDate`, `byPackage`, `byCustomer`, `byPaymentMethod`
- 30-second `CacheService` TTL with auto-invalidate hooks
- Client receives ready-to-render objects (eliminates $O(n)$ loops)
- Load time: 1.3-4.8s (first) → <200ms (cache hit)

Export Formats:

- CSV (transactions, daily summary)
- PDF (thermal-ready receipts, detailed reports)

Key Functions:

- `getReportData(token, start, end)` - Pre-aggregated report data
 - `getReportAndKasData(token, start, end)` - Bundle report + cash
 - `getDashboardBundle(token)` - 1-RPC dashboard hydration
-

Data Flow

Login & Dashboard Initialization

1. User submits credentials (login page)
↓
2. `login(username, password)`
 - Read users sheet
 - Verify salted hash
 - Generate UUID token (TTL 8h)
 - Store in CacheService↓
3. Client receives { success, token, role }
↓
4. `getDashboardBundle(token)`
 - Parallel fetch: transactions (300), packages, settings, customers, activeShift
 - `_perf` wrapper tracks per-child timing↓
5. Client renders dashboard
 - Persist transactions → `sessionStorage`
 - Persist settings → `localStorage` (TTL 1h)
 - Hydrate RAM: `allTransactions`, `allCustomers`

Transaction Creation Flow

1. Kasir selects customer + items + payment method
↓
2. `createTransaction(token, data)`
 - `LockService.lock()` → atomic write
 - Validate: totals, discounts, payment method
 - Lookup prices via cached price map (5m TTL)
 - Batch write to transactions sheet
 - Tag `shift_id` (current active shift)
 - Invalidate cache: transactions, kas, report↓
3. Server returns { success, id, total }
↓
4. Client displays thermal receipt + WhatsApp button
↓
5. Auto-refresh: kas card, history section, dashboard stats

Shift Close Flow

1. Kasir clicks "Tutup Shift"
↓
2. `closeShift(token, shiftId, catatan)`

```
- LockService.lock()
- computeShiftSummary_()
  - Scan transactions with shift_id
  - Aggregate by payment method (Tunai/QRIS/Transfer)
  - Aggregate by payment type (DP/Pelunasan)
  - Aggregate expenses by category
- Persist breakdown_json (column 15)
- Calculate saldo_akhir = modal_awal + tunai - pengeluaran
- Update shifts sheet
- Invalidate cache: shifts, kas, report
↓
3. Server returns { success, summary, saldo_akhir }
↓
4. Client displays modal "Shift Selesai" with breakdown
↓
5. Kasir confirms → redirect to dashboard
```

Performance Optimization

Optimization Pillars

Pillar	Implementation	Impact
RPC Bundling	getDashboardBundle (1 call vs 5 serial)	Save 2-8s per login
Warmup Trigger	5-min V8 keep-alive	Cold start 3-8s → ~500ms
Range-Bounded Reads	Hot paths read window, not full sheet	60-80% latency reduction
Price Map Cache	Lookup via id→price map (RAM + 5m TTL)	Save 200-400ms/transaction
Batch Writes	setValues([rows]) vs multiple setValue()	400-600% faster writes
Persisted Summary	shifts sheet stores saldo + breakdown_json	History without recompute
Active Shifts Cache	15-second CacheService TTL	Safe aggressive polling
Lazy-Load Assets	ApexCharts (250KB) + jsPDF (150KB) on-demand	Save 400KB initial load
Smart Polling	Pause when tab hidden, resume on focus	Reduce GAS quota usage
Report Pre-Aggregation	Server computes byDate/byPackage/etc	0.8-1.5s load (vs 2-5s)

4-Layer Caching Strategy

Layer 1: Server CacheService

- Session tokens: 8 hours
- Customers: 2 minutes (max 1500 rows)
- Packages: 10 minutes
- Price map: 5 minutes
- Settings: 5 minutes
- Report data: 30 seconds
- Active shifts: 15 seconds

Layer 2: Memory Cache

- Price map per execution (in-process)
- Temporary aggregations during shift close

Layer 3: Client RAM

- allTransactions (1 minute TTL)
- allCustomers (2 minute TTL)
- allPackages (5 minute TTL)

Layer 4: Persistent Client

- sessionStorage: transactions (hydration on page reload)
- localStorage: settings (1 hour TTL)
- IndexedDB: transactions (persistent cache across sessions)

Profiling Instrumentation

Server-Side:

```
// Wrap hot paths with _perf()
_perf("getDashboardBundle", () => {
  // function body
});

// Run benchmark (admin only)
runPerfBenchmark(token)
// Returns: { fnName: { ms, ok, size }, ... }
```

Client-Side:

```
// From DevTools console
perfStats()    // Table: p50/p95/max/avg per function
perfClear()    // Reset log
window.PERF_OFF = true // Disable logging
```

Logging:

- Server: Stackdriver Executions (all calls) + perf_logs sheet (1 of 5 sample)
- Auto-rotate: perf_logs (1000 rows), error_logs (500 rows)

Security & Reliability

Security Measures

1. Token-Based Session

- UUID tokens with 8-hour TTL
- Rolling refresh on each request
- Stored in CacheService (server-side)

2. Password Security

- Salted hash: `salt:hash` format
- Auto-migration from legacy formats
- Rate limiting: 5 failures → 15-min lockout

3. Admin Guards

- `validateAdminSession_()` on sensitive endpoints
- Protected: `deleteTransaction`, `forceCloseShift`, `getUsersList`, `runPerfBenchmark`

4. Server-Side Validation

- Total, discount, status re-validated on server
- Prevents DevTools manipulation
- Price lookup via cached map (not client-provided)

5. DB_ID Protection

- Stored in Script Properties (not hardcoded)
- Auto-migration on first run

Reliability Measures

1. Atomic Writes

- `LockService` on all critical operations
- Prevents race conditions in multi-user scenarios

2. Defensive Dashboard

- Per-block try/catch in `setupDashboard`
- One function error doesn't crash entire dashboard

3. Error Logging

- Silent logging to `error_logs` sheet
- Timestamp, function name, message, context
- Auto-rotate (500 rows)

4. Auto-Backup

- Daily 02:00 backup to Google Drive
- Duplicates entire spreadsheet

5. Auto-Close Fail-Safe

- Daily 23:50 trigger closes abandoned shifts
- Prevents data loss from forgotten close

Database Schema

Sheet: users (4 columns)

Column	Type	Notes
username	string	Primary key, case-insensitive
password	string	Salted hash: <code>salt:hash</code>

role	enum	admin OR kasir
nama	string	Display name

Sheet: packages (7 columns)

Column	Type	Notes
id	UUID	Primary key
nama_paket	string	Service name
harga	int	Price per unit
durasi_hari	int	Estimated days
satuan	string	Kg / Pcs / Set
kategori	string	Grouping tag
status	enum	Aktif OR Nonaktif

Sheet: transactions (23 columns)

#	Column	Type	Notes
1	id	UUID	Primary key
2	tanggal	ISO datetime	Creation timestamp
3	customer	string	Customer name
4	paket	string	Main service (legacy)
5	berat	float	Total weight (legacy)
6	total	int	Grand total
7	status	enum	Proses/Selesai/Diambil
8	kasir	string	Cashier username
9	whatsapp	string	Customer phone
10	satuan	string	Default unit
11	estimasi_selesai	ISO datetime	Target completion
12	metode_pembayaran	string	Tunai/Transfer/QRIS
13	status_pembayaran	enum	Lunas/Belum Lunas
14	metode_pelunasan	string	Settlement method
15	(reserved)	—	—
16	catatan	string	Cashier notes

17	terbayar	int	Total paid
18	items_json	JSON	Multi-item array
19	tanggal_pelunasan	ISO datetime	Settlement date
20	nominal_dp	int	Down payment
21	nominal_pelunasan	int	Settlement amount
22	shift_id	UUID	FK → shifts.id (creation)
23	pelunasan_shift_id	UUID	FK → shifts.id (settlement)

Sheet: shifts (15 columns)

#	Column	Type	Notes
1	id	UUID	Primary key
2	kasir	string	Cashier username
3	nama_kasir	string	Display name
4	waktu_mulai	ISO datetime	Shift start
5	waktu_selesai	ISO datetime	Shift end
6	modal_awal	int	Initial cash
7	total_transaksi	int	Transaction total (persisted)
8	total_tunai	int	Cash received (persisted)
9	total_non_tunai	int	Non-cash received (persisted)
10	jumlah_order	int	Order count (persisted)
11	status	enum	Aktif/Selesai/Force-Closed
12	catatan	string	Close notes
13	total_pengeluaran	int	Expenses (persisted)
14	saldo_akhir	int	modal + tunai - pengeluaran
15	breakdown_json	JSON	Detailed breakdown (v2.3+)

Supporting Sheets

Sheet	Columns	Purpose
settings	key, value	App config (store name, logo, WA template)
customers	id, nama, whatsapp, terakhir_order	Customer master
kas_awal	tanggal, nominal, kasir	Daily initial cash

pengeluaran	id, tanggal, keterangan, kategori, jumlah, kasir	Expense log
error_logs	Waktu, Fungsi, Pesan, Context	Error audit (rotate 500)
perf_logs	timestamp, fn, ms, ok	Performance sample (rotate 1000)

API Reference

Authentication & Session

Endpoint	Access	Description
login(username, password)	Public	Verify credentials, issue UUID token
validateSession_(token)	Internal	Check token validity, refresh TTL
validateAdminSession_(token)	Internal	Guard admin-only endpoints

Transactions

Endpoint	Access	Description
getTransactions(token)	User	Get 300 latest transactions
getTransactionsIncremental(token, lastHash)	User	Incremental sync with hash comparison
createTransaction(token, data)	User	Create transaction (Lock + batch write)
updateTransactionStatus(token, id, status)	User	Change status (Proses/Selesai/Diambil)
lunasDanAmbil(token, id, metode)	User	Settlement + tag pelunasan_shift_id
deleteTransaction(token, id)	Admin	Delete transaction (audited)

Customers

Endpoint	Access	Description
getCustomers(token)	User	Get customer list (max 1500, cached 2m)
addCustomerData(token, nama, whatsapp)	User	Add new customer
updateCustomerData(token, id, nama, whatsapp)	User	Update customer
deleteCustomerData(token, id)	User	Delete customer

Packages (Services)

Endpoint	Access	Description
getPackages(token)	User	Get service catalog (cached 10m)
addPackage(token, data)	Admin	Add new service
updatePackage(token, id, data)	Admin	Update service

updatePackageStatus(token, id, status)	Admin	Toggle Aktif/Nonaktif
deletePackage(token, id)	Admin	Delete service

Cash Management

Endpoint	Access	Description
getKasHarian(token, tanggalStr)	User	Daily cash with scope resolution
saveUangAwal(token, nominal, dateStr)	User	Record initial cash
savePengeluaran(token, keterangan, kategori, jumlah, dateStr)	User	Log expense
deletePengeluaran(token, id)	User	Delete expense
getKasPeriode(token, start, end)	Admin	Multi-day cash recap

Shift Management

Endpoint	Access	Description
getActiveShiftAPI(token)	User	Get user's active shift
getAllActiveShifts(token)	Admin	All active shifts (15s cache)
openShift(token, modalAwal)	User	Begin shift, sync kas_awal
closeShift(token, shiftId, catatan)	User	End shift + persist breakdown
forceCloseShift(token, shiftId, catatan)	Admin	Force-close abandoned shift
getShiftHistory(token, start, end)	Admin	Shift history (max 500 rows)
recomputeShiftBreakdown(token, shiftId)	Admin	Lazy-migrate pre-v2.3 shifts
autoCloseExpiredShifts()	Trigger	Daily 23:50 fail-safe

Reports & Dashboard

Endpoint	Access	Description
getDashboardBundle(token)	User	1-RPC bundle: trx, packages, settings, customers, shift
getReportData(token, start, end)	Admin	Pre-aggregated report data (30s cache)
getReportAndKasData(token, start, end)	Admin	Bundle: report + cash data
aggregateReportData_(transactions)	Internal	Server-side aggregation helper

User Management

Endpoint	Access	Description
getUsersList(token)	Admin	List all users

saveUserAccount(token, username, password, role, nama)	Admin	Add/edit user
deleteUserAccount(token, username)	Admin	Delete user

Settings

Endpoint	Access	Description
getSettings()	Public	Get app settings
saveSettingsConfig(token, dataObj)	Admin	Update settings

Maintenance & Profiling

Endpoint	Access	Description
setupDatabase()	Manual	Initialize sheets (idempotent)
setupAllTriggers()	Manual	Install warmup + auto-close + backup
runPerfBenchmark(token)	Admin	Diagnose hot path latency
warmup()	Trigger	V8 keep-alive (every 5 min)
dailyBackup()	Trigger	Backup DB to Drive (02:00)
disablePerfLogging()	Admin	Disable server profiling

Cache Invalidation Hooks

Function	Invalidates
invalidateTransactionsCache_()	transactions, report data
invalidateShiftCache_()	active shifts, shift history
invalidateKasCache_()	kas harian, kas periode
invalidateReportCache_()	report data, kas periode

Deployment Guide

Prerequisites

- Node.js $\geq$ 18.x and npm
- Google account with Apps Script access
- (Optional) Tailwind CLI for CSS rebuild

Quick Start

```
# 1. Clone repository
git clone https://github.com/nndda-rzn/kucucikan-laundry-gas.git
cd kucucikan-laundry-gas
```

```
# 2. Install clasp globally
npm install -g @google/clasp

# 3. Login to Google
clasp login

# 4a. Connect to existing Apps Script project
clasp clone <YOUR_SCRIPT_ID>

# 4b. OR create new project
clasp create --type webapp --title "Kucucikan Laundry POS"

# 5. Push source to server
clasp push

# 6. Create new deployment version (REQUIRED for /exec URL)
clasp deploy
```

Post-Deploy Configuration

Run these functions **once** from Apps Script Editor:

Function	Purpose	Frequency
setupDatabase	Initialize sheets & migrate columns (idempotent)	Once
setupAllTriggers	Install warmup + auto-close + auto-backup	Once
runPerfBenchmark	Diagnose hot path latency	On-demand

Installed Triggers

Trigger	Frequency	Function	Purpose
warmup	Every 5 minutes	V8 keep-alive	Prevent 3-8s cold start
autoCloseExpiredShifts	Daily 23:50	Shift fail-safe	Close abandoned shifts
dailyBackup	Daily 02:00	DB backup	Duplicate spreadsheet to Drive

Development Workflow

```
# Edit files locally
code .

# Sync to GAS
clasp push

# Create new deployment version (for /exec URL)
clasp deploy

# OR test via /dev URL (always uses HEAD)
```

```
clasp open  
# → Deploy → Test deployments → copy URL
```

Important: `clasp push` alone does NOT update the `/exec` production URL. A new deployment version must be created for GAS to serve the latest code. Use `/dev` URL for rapid iteration during development.

Build Tailwind (Optional)

```
npm install -D @tailwindcss/cli  
node build-tailwind.js  
clasp push
```

Troubleshooting

Common Issues

1. Cold Start Latency (3-8 seconds)

Symptom: First request after idle period is very slow.

Solution:

- Ensure `warmup` trigger is installed: `setupAllTriggers()`
- Verify trigger in Apps Script Editor → Triggers
- Warmup runs every 5 minutes to keep V8 engine warm

2. "Session Expired" Error

Symptom: User gets logged out unexpectedly.

Causes:

- Token TTL (8 hours) exceeded
- CacheService eviction under load
- Multiple tabs with different sessions

Solution:

- Re-login to get fresh token
- Check for multiple active sessions

3. Shift Not Closing Properly

Symptom: Shift remains "Aktif" after close attempt.

Causes:

- LockService timeout (concurrent operations)
- Network interruption during close

Solution:

- Admin can force-close via Manajemen Shift section
- Check `error_logs` sheet for details
- Auto-close trigger runs at 23:50 as fail-safe

4. Transactions Not Syncing

Symptom: New transactions not appearing on other devices.

Causes:

- Client cache not invalidated
- Polling paused (tab in background)

Solution:

- Bring tab to foreground (triggers immediate refresh)
- Manual refresh via pull-to-refresh or F5
- Check if polling is active (60-second interval)

5. Report Loading Slowly

Symptom: Laporan section takes >3 seconds to load.

Causes:

- Cache miss (first load or after invalidation)
- Large date range with many transactions

Solution:

- Wait for cache to warm up (30s TTL)
- Use smaller date ranges for initial load
- Check perf_logs for server timing

6. WhatsApp Button Not Working

Symptom: WhatsApp confirmation doesn't open.

Causes:

- Invalid phone number format
- Popup blocked by browser

Solution:

- Ensure phone number starts with +62 or 08
- Allow popups for the domain
- Check WA template in Settings

Diagnostic Tools

Server-Side

```
// Run from Apps Script Editor (admin token required)
runPerfBenchmark(token)
// Returns: { fnName: { ms, ok, size }, ... }

// Check Stackdriver logs
// Apps Script Editor → Executions → filter "PERF"
// Shows: [PERF][getDashboardBundle] 1234ms with child timings
```

```
// Check perf_logs sheet
// Sample 1 of 5 calls stored, auto-rotate 1000 rows
```

Client-Side

```
// From DevTools console
perfStats() // Table: p50/p95/max/avg per function
perfClear() // Reset log

// Disable profiling
window.PERF_OFF = true

// Check IndexedDB cache
trxDB.load().then(console.log)

// Check localStorage
localStorage.getItem('settings_cache')
localStorage.getItem('settings_cache_time')
```

Error Log Analysis

Location: Sheet `error_logs`

Columns:

- Waktu: Timestamp
- Fungsi: Function name where error occurred
- Pesan: Error message
- Context: Additional context (user, parameters)

Common Patterns:

- `LockService timeout` → Concurrent write conflict
- `Invalid token` → Session expired
- `Sheet not found` → Run `setUpDatabase()`
- `Quota exceeded` → GAS daily limits reached

Performance Benchmarks

Expected Latencies (warm):

Operation	Target	Acceptable
Login	<1s	<2s
Dashboard load	<1.5s	<3s
Create transaction	<800ms	<1.5s
Close shift	<1s	<2s
Report load (cached)	<200ms	<500ms
Report load (fresh)	<1.5s	<3s

If exceeding acceptable:

1. Check warmup trigger is active
 2. Review perf_logs for slow functions
 3. Verify cache TTLs are appropriate
 4. Consider reducing date range for reports
-

Version History

v2.8 — Report Generation Optimization (2026-05)

- Server-side CacheService (30s TTL) for getReportData & getKasPeriode
- Pre-aggregation on server: byDate, byPackage, byCustomer, byPaymentMethod
- Auto-invalidate hooks on transaction/shift/kas mutations
- Load time: 1.3-4.8s → 0.8-1.5s (first), <200ms (cache hit)

v2.7 — Multi-Device Realtime Sync (2026-05)

- Polling coverage expanded to section-kas and section-shift
- Visibility-aware: pause when hidden, instant refresh on focus
- 60-second cycle per relevant section

v2.6 — Total Penerimaan Card (2026-05)

- Card "Total Penerimaan" auto-calculated (previously hardcoded 0)
- New fields: total_penerimaan, penerimaan_tunai, penerimaan_non_tunai
- Scope-aware logic (shift/date/none)
- Helper refreshKasIfVisible() after mutations

v2.5 — UX/A11y Polish (2026-05)

- 4 shift modals with role="dialog" + aria-modal
- WCAG AA contrast (text-slate-500 for small labels)
- Default landing role-aware (kasir → Transaksi)
- Search & filter in Riwayat Shift tab
- Progressive disclosure 2-layer cards
- Loading state helpers: setBtnLoading, renderListSkeleton

v2.4 — Role-Aware Cash Management (2026-05)

- getKasHarian returns scope per request (shift/date/none)
- Frontend applyKasScope with dynamic labels
- Auto-refresh kas on openShift/closeShift

v2.3 — Admin Shift Management (2026-05)

- Section "Manajemen Shift" admin-only
- getAllActiveShifts with 15s cache + auto-invalidate
- computeShiftSummary_ with breakdown per method & category
- Persist breakdown_json (column 15)
- Endpoint recomputeShiftBreakdown for lazy migration

v2.2 — Performance & Profiling (2026-05)

- Profiling instrumentation: _perf wrapper, runPerfBenchmark
- Warmup trigger (cold start 3-8s → ~500ms)
- Range-bounded reads on hot paths

- Price map cache for createTransaction
- Smart polling with visibilitychange
- sessionStorage hydration for transactions

v2.1 — Shift Management (2026-05)

- Full shift lifecycle: open/close/force-close/auto-close
- Attribution via pelunasan_shift_id column
- Tab Rekap Shift in admin reports
- LF-only line endings via .gitattributes

v2.0 — Performance Wave (2026-05)

- getDashboardBundle (1 RPC vs 4 serial)
- Lazy-load ApexCharts & jsPDF
- 3-layer caching strategy
- Date-bounding (300 latest transactions)
- Optimistic UI for package status toggle
- Batch sheet operations

v1.x — Foundation

- Daily cash management
- User management with RBAC
- WhatsApp integration
- Auto-save draft transactions
- LockService on all writes
- Auto-backup to Drive
- Tailwind v4 UI overhaul

Role Access Matrix

Feature	Admin	Kasir
Create/edit transactions	✓	✓
Delete transactions	✓	—
Customer management	✓	✓
Package management (CRUD)	✓	read-only
Open/close own shift	bypass	✓
Force-close any shift	✓	—
Section Manajemen Shift	✓	—
Shift history & recompute	✓	—
User management	✓	—
Store settings & WA template	✓	—
Reports & export PDF/CSV	✓	—

Profiling (runPerfBenchmark)	✓	—
------------------------------	---	---

Default Credentials

Username	Password	Role
admin	admin123	admin
kasir	kasir123	kasir

 **Important:** Change passwords immediately after first login via **Manajemen Pegawai**.

File Structure

kucucikan-laundry-gas/	
├─ appsscript.json	# GAS manifest (timezone, runtime, webapp config)
├─ Kode.gs	# Backend: 70+ functions (auth, CRUD, shift, profiling)
├─ index.html	# Main HTML template (includes all components)
├─ JavaScript.html	# Client-side JS (state, RPC, UI logic)
├─ CSS.html	# Custom CSS (animations, print styles)
├─ Tailwind.html	# Generated Tailwind CSS v4.2.3
├─ package.json	# npm dependencies (tailwindcss)
├─ build-tailwind.js	# Tailwind build script
├─ .clasp.json	# clasp configuration (script ID)
├─ .claspignore	# Files excluded from push
├─ .gitattributes	# LF-only line endings
├─ README.md	# Project overview & quick start
├─ SYSTEM_DOCUMENTATION.md	# This file
└─ docs/	
└─ dokumentasi.html	# Printable HTML documentation

Contact & Support

- **Repository:** <https://github.com/nndda-rzn/kucucikan-laundry-gas>
- **Issues:** <https://github.com/nndda-rzn/kucucikan-laundry-gas/issues>

Built with care for laundry operations that demand speed, accuracy, and accountability.

Last updated: 2026-05-19 | Version 2.8